Université des Sciences et de la Technologie Houari Boumediène

Faculty of Mathematics
Department of Operations Research

Optimization Using Specific Swarms

PSO (Discrete Version) Applied to the Traveling Salesman Problem

---

Master M1 ROMARIN - Heuristic and Metaheuristic Methods

GitHub Repository:
https://github.com/Adel-dev2/Particle-Swarm-Optimization

Group Members

- BEKKA Rayan
- BOUANANE Adel
- NASRI Samir
- DJOUZA Abdelkayoum
- FILALI Ishak
- KHERROUF Younes

Supervised by: **DR. M. YAGOUNI**

May 6, 2026

Abstract

This report presents a comprehensive study and implementation of Particle Swarm Optimization (PSO) applied to the Traveling Salesman Problem (TSP) .

PSO is a population-based metaheuristic inspired by the collective behavior of bird flocks and fish schools, introduced by Kennedy and Eberhart in 1995. The TSP, a canonical NP-hard combinatorial optimization problem, requires finding the shortest Hamiltonian cycle through a given set of cities. Since standard PSO operates natively in continuous space while TSP solutions are discrete permutations, this report covers two complementary approaches: (1) the classical Discrete PSO (DPSO) formulation using redefined discrete operators ($\ominus$, $\oplus$, $\otimes$), and (2) the Smallest Position Value (SPV) rule that converts real-valued particle positions into valid tours via the `argsort` operator.

The implemented framework, written in Python, adopts a modular Strategy design pattern, providing interchangeable components for initialization, inertia weight scheduling, swarm topology, velocity update rules, and boundary handling. A 2-opt local search is hybridized with the global best solution at configurable intervals.

Experiments are conducted on benchmark instances from the TSPLIB library (eil51, berlin52, eil76, kroA100, ch130, ch150, kroC100) over 30 independent runs each. Results are compared against known optimal solutions (BKS) using the Relative Error (ER) metric.

Last but not least, we worked on improving the algorithm using reinforcement Learning. We trained the swarm as an agent that was treated on 1000 episodes, each episode the agent learns to choose the right set of parameters (ω, c1 and c2) based on its state (Diversity, stagnation and current iteration). After training we acheived modest improvements on the used benchmarks considering the limited time and hardware available.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 General Context

Combinatorial optimization is omnipresent in the real world. As a concept, it has existed since the beginning of time and has been even more apparent in the modern world due to its ties to a lot of fields such as logistics, commerce, and many others.

1.2 Combinatorial Optimization: Definition and Challenges

1.2.1 Definition

Combinatorial Optimization is a field born from the discipline of Operations Research and aims to find a solution that optimizes a certain criterion (cost, distance, time, etc.) among a finite yet astronomically large set of feasible solutions.

Example: Minimize $f(x)$ subject to constraints, with $x \in S$, where S is a finite feasible solution set.

1.2.2 Its Main Challenge

The difficulty born from the discipline is that the size of the set of possible solutions S grows combinatorially (often factorially or exponentially), which can make exhaustive enumeration out of reach.

And that is where computational complexity theory comes in, as it lays a theoretical foundation to understand why certain problems can be simple to state yet extraordinarily hard to solve, resisting efficient algorithmic solutions.

1.3 Classes of Problems

Problems can be categorized into 4 main classes depending on the time it takes to solve them.

Class P: Problems solvable in polynomial time (examples: sorting, shortest path). These are considered "tractable."

Class NP: Problems for which a candidate solution can be verified in polynomial time, but for which no polynomial-time solving algorithm is currently known.

NP-hardness: A problem is NP-hard if every problem in NP can be reduced to it in polynomial time. In plain terms: if a polynomial-time algorithm were found for such a problem, every NP problem would become polynomially solvable (P = NP).

NP-completeness: A problem that is both NP-hard and belongs to NP.

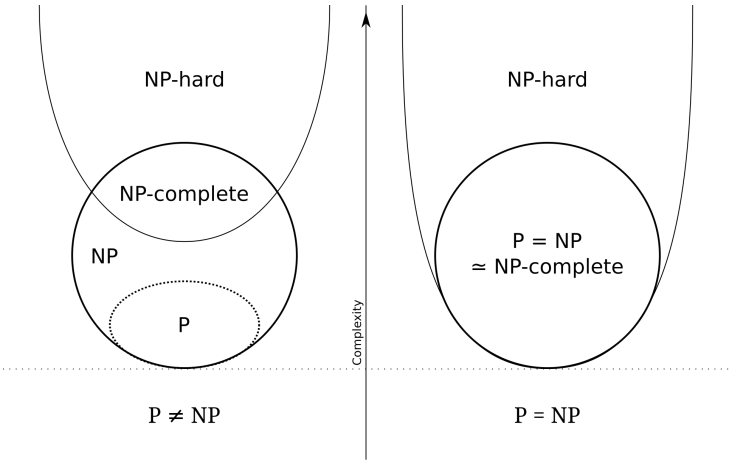

Figure 1.1: Complexity Classes

Faced with these problems, three broad strategies emerge: exact algorithms, heuristics, and metaheuristics.

1.4 The Travelling Salesman Problem (TSP)

1.4.1 Definition and Historical Significance

The Travelling Salesman Problem is one of the most extensively studied problems in combinatorial optimization, as it has a very deceptively simple statement but a profound algorithmic depth. It was formally studied as early as the 1930s (Karl Menger), and became the canonical benchmark for exact methods (Dantzig et al., 1954; the Concorde

Solver), heuristics (2-opt, 3-opt, Lin–Kernighan), and metaheuristics (simulated annealing, genetic algorithms, ant colony optimization).

1.4.2 Formal Definition

Definition 1.1 (Traveling Salesman Problem)**.** Given a complete weighted graph $G = (V, E, d)$ where $V = \{1, 2, \ldots, n\}$ is a set of n cities, $E = V \times V$ is the set of edges, and $d : E \to \mathbb{R}^+$ is a distance (cost) function, the TSP seeks a permutation $\pi \in \mathcal{S}_n$ minimizing the total tour length:

$$f(\pi) = \sum_{k=1}^{n-1} d_{\pi_k, \pi_{k+1}} + d_{\pi_n, \pi_1} \tag{1.1}$$

where d_{ij} is the distance between cities i and j.

The travelling salesman has to find a tour that allows him to visit each city exactly once and only once, returning to the starting city, while minimizing the total distance traveled (or the total cost respectively) which graphically translates to finding the minimum cost Hamiltonian circuit.

In the TSPLIB benchmark library, the `EUC_2D` distance convention is used:

$$d_{ij} = \text{round}\left(\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}\right) \tag{1.2}$$

where (x_k, y_k) are the Euclidean coordinates of city k.

1.4.3 MATHEMATICAL FORMULATION

The following formulation was given by Dantzig, Fulkerson and Johnson in 1954. Given:
— $G = (X, U)$, where:

$$|X| = n; \quad U = (i, j); , i, j \in X, , i \neq j; \quad |U| = \frac{n(n-1)}{2} \tag{1.3}$$

— Distance matrix $D = (dij), , i, j \in X$. Decision variables:

$$xij = \begin{cases} 1 & \text{if } (i, j) \in \text{cycle} \\ 0 & \text{otherwise} \end{cases} \tag{1.4}$$

The associated linear program:

$$\min \sum i = 1^n \sum j=1 \ i \neq j^n dijxij \tag{1.5}$$

$$\sum i=1 \ i \neq j^n xij = 1, \quad \forall j \in 1, \ldots, n \tag{1.6}$$

$$\sum j=1 \ j \neq i^n xij = 1, \quad \forall i \in 1, \ldots, n \tag{1.7}$$

$$\sum i, j \in Sxi, j \leq |S| - 1, \quad S \subset V, \quad 2 \leq |S| \leq n - 2 \tag{1.8}$$

$$x_{i,j} \in 0, 1 \quad i, j = 1, \ldots, n, ; i \neq j \tag{1.9}$$

Relation (1.3): the objective function minimizes the sum of the unit costs of the arcs included in the circuit. Constraints (1.4)–(1.5) express the fact that each city must be visited exactly once. These constraints alone are not sufficient to describe valid tours, which is why constraint (1.6) — known as the *subtour elimination constraints* — is introduced. This constraint implies that no tour of size less than $|V|$ exists in the solution. The latter constraint enforces the connectivity of the tour. A graph is said to be connected if there exists a path between every pair of nodes. The last constraint (1.7) defines the domain of the decision variables. We refer to *linear relaxation* as the process of relaxing this last constraint.

1.4.4 Main Variants of the Problem

Symmetric TSP: $d(i, j) = d(j, i)$, which is the most classical setting.

Asymmetric TSP: The distances are not symmetric (Example: directional transport costs).

Metric TSP: It satisfies the triangle inequality and is frequently used in approximation proofs.

1.4.5 Complexity of TSP and Search Space Growth

The Travelling Salesman Problem is NP-hard in its general form and is proven by the reduction from the Hamiltonian circuit problem. Its decision version is NP-complete.

The search space has cardinality $|\mathcal{S}_n| = n!$ in the asymmetric case, reduced to $(n-1)!/2$ in the symmetric case. Table 1.1 illustrates how this quantity grows:

Table 1.1: Exponential growth of TSP search space

Cities n	**Number of tours** $(n-1)!/2$	**At** 10^{12} **tours/s**
10	181,440	$<$ 1 microsecond
20	$\approx 6 \times 10^{16}$	$\approx$ 2 years
51	$\approx 1.5 \times 10^{62}$	$\approx 10^{50}$ years
130	$\approx 10^{217}$	$\gg$ age of universe

1.4.6 Graphical Example

A travelling salesman has to find an optimal tour between 6 French cities:

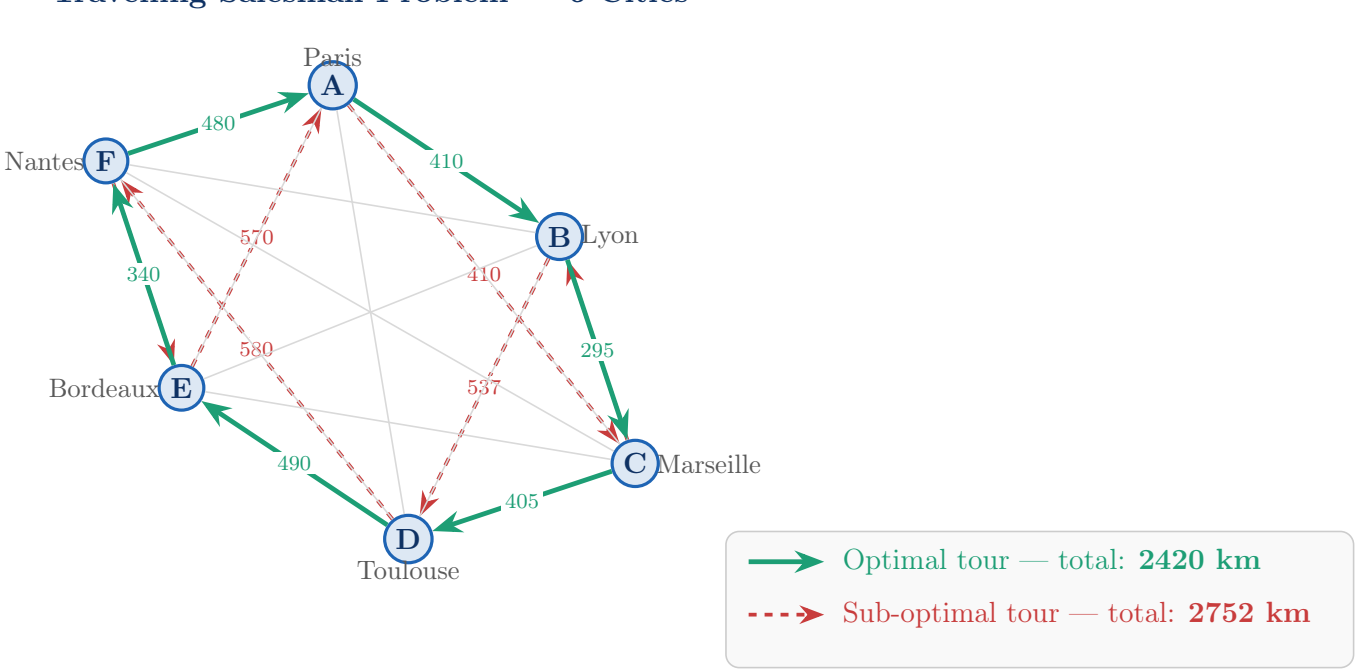

Figure 1.2: Optimal vs. sub-optimal TSP tour on 6 French cities

1.4.7 Why Metaheuristics?

While exact solvers such as Concorde have solved instances with $n \approx 85{,}900$ cities, they require hours of computation time. For real-time applications or large-scale logistics, metaheuristics offer the only viable approach, trading optimality guarantees for computational efficiency.

Chapter 2

State of the Art on Particle Swarm Optimization

2.1 Historical Background

2.1.1 Biological Inspiration: Reynolds' Boids Model (1987)

The conceptual foundation of PSO was laid by Craig Reynolds in 1987 with his *Boids* model, which demonstrated that the complex, coordinated movement of bird flocks and fish schools can emerge from three simple local rules:

1. **Separation:** Avoid crowding neighbors (short-range repulsion).
2. **Alignment:** Steer toward the average heading of neighbors.
3. **Cohesion:** Steer toward the average position of neighbors (long-range attraction).

No global coordinator exists; yet globally coherent, adaptive behavior emerges.

2.1.2 Kennedy and Eberhart (1995): Birth of PSO

The PSO method was described in two papers co-authored by the electrical engineer Russell C. Eberhart and the social psychologist James Kennedy in 1995. The technique is derived from a computational study and simulation of a simplified social model of bird flocks seeking for food, with the intent to graphically simulate the graceful and unpredictable choreography of a bird flock. From this initial objective, the concept evolved into a simple and efficient optimization algorithm.

Kennedy and Eberhart equipped each individual (called a *particle*) with two memory components:

- **Personal best (p_i):** The best solution the particle has personally visited (*cognitive memory*).
- **Global best (g):** The best solution found by any particle in the entire swarm (*social knowledge*).

The PSO method is based on the premise that the knowledge lies not only in the social sharing of information among generations but also between elements of the same generation. Although PSO shares some characteristics with other population-based models such as Genetic Algorithms (GA), it has the benefit of being relatively simple and easy to implement.

2.2 General Principle and Equations of Motion

2.2.1 The Particle

The PSO computational method aims to optimize a problem iteratively, starting by initializing a set (or population) of candidate solutions, called a *swarm of particles*. Each particle $i \in \{1, \ldots, N\}$ at iteration t is characterized by:

- **Position $\mathbf{x}_i^t \in \mathbb{R}^D$**: the candidate solution represented by this particle.
- **Velocity $\mathbf{v}_i^t \in \mathbb{R}^D$**: the direction and magnitude of the particle's movement.
- **Personal best $\mathbf{p}_i = \arg\min_{\mathbf{x}_i^\tau, \tau \leq t} f(\mathbf{x}_i^\tau)$**: the best position particle i has ever visited.

Positions and velocities are updated at each iteration based on:

1. The particle's own best known position (personal best, pbest).
2. The best position discovered by any particle in the swarm (global best, gbest).

The **global best $\mathbf{g}$** of the swarm is:

$$\mathbf{g} = \underset{\mathbf{p}_j, j \in \{1,\ldots,N\}}{\arg\min} f(\mathbf{p}_j) \tag{2.1}$$

2.2.2 The Velocity Update Equation

The velocity of particle i at iteration $t+1$ is updated according to:

$$\mathbf{v}_i^{t+1} = \underbrace{\omega \mathbf{v}_i^t}_{\text{inertia}} + \underbrace{c_1 \mathbf{r}_1^t \odot (\mathbf{p}_i - \mathbf{x}_i^t)}_{\text{cognitive term}} + \underbrace{c_2 \mathbf{r}_2^t \odot (\mathbf{g} - \mathbf{x}_i^t)}_{\text{social term}} \tag{2.2}$$

where:

- $\omega \in [0, 1]$ is the **inertia weight**, controlling momentum;
- c_1 and c_2 are real acceleration coefficients known as the cognitive and social weights;
- $\mathbf{r}_1^t, \mathbf{r}_2^t \in [0, 1]^D$ are independent random vectors sampled i.i.d. from $U(0, 1)$;
- $\odot$ denotes element-wise (Hadamard) multiplication.

The three terms serve distinct roles:

1. **Inertia:** Preserves the particle's current trajectory. High ω promotes exploration; low ω promotes exploitation.
2. **Cognitive:** Pulls the particle toward its own historical best embodying personal experience.
3. **Social:** Pulls the particle toward the swarm's collective best embodying shared knowledge.

2.2.3 The Position Update Equation

The position is updated at each iteration by:

$$\vec{x}_i^{t+1} = \vec{x}_i^t + \vec{V}_i^{t+1}$$

The initial best personal position is the particle's initial position $\vec{p}_i^0 = \vec{x}_i^0$. At iteration $t+1$ it is calculated as:

$$\mathbf{y}_i(t+1) = \begin{cases} \mathbf{y}_i(t) & \text{if } f(\mathbf{x}_i(t+1)) \geq f(\mathbf{y}_i(t)) \\ \mathbf{x}_i(t+1) & \text{if } f(\mathbf{x}_i(t+1)) < f(\mathbf{y}_i(t)) \end{cases}$$

where y_i is the pbest and $f : \mathbb{R}^{n_x} \to \mathbb{R}$ is the fitness function.

The global best position, $\hat{y}(t)$, at time step t, is defined as:

$$\hat{y}(t) \in \{y_0(t), \ldots, y_n(t)\}, \quad \hat{y}(t) = \min\{f(y_0(t)), \ldots, f(y_n(t))\}$$

where n_s is the total number of particles in the swarm.

The gbest PSO is summarized in Algorithm 1.

Algorithm 1 gbest PSO

```
Create and initialize an n_x-dimensional swarm;
repeat
    for each particle i = 1, ..., n_s do                    ▷ set personal best
        if f(x_i) < f(y_i) then
            y_i = x_i;
        end if                                              ▷ set global best
        if f(y_i) < f(ŷ) then
            ŷ = y_i;
        end if
    end for
    for each particle i = 1, ..., n_s do
        update velocity
        update position
    end for
until stopping condition is true
```

2.2.4 The Stochastic Attractor

Geometrically, the velocity update steers each particle toward a probabilistic combination of $\mathbf{p}_i$ and $\mathbf{g}$. The expected attractor (weighted centroid) is:

$$\mathbf{G}_i = \frac{c_1\mathbf{p}_i + c_2\mathbf{g}}{c_1 + c_2} \tag{2.3}$$

When $c_1 = c_2$, this centroid is exactly the midpoint between $\mathbf{p}_i$ and $\mathbf{g}$.

2.3 PSO Improvements

2.3.1 The Inertia Weight Parameter

In 1998, Shi and Eberhart introduced the notion of the inertia weight ω. This coefficient controls the local and global search ability, determining how much influence the previous velocity should have on the current particle's movement. With this parameter, the velocity update equation becomes (Eq. 2.2).

Van den Bergh stated a strong relationship between c_1, c_2 and ω, modelled by:

$$\omega > \frac{1}{2}(c_1 + c_2) - 1.$$

Constant Inertia Weight

$\omega(t) = \omega_0$ for all t. Typically $\omega_0 = 0.729$. Serves as a baseline.

Linear Decreasing Inertia Weight (LDIW)

$$\omega(t) = \omega_{\max} - \frac{\omega_{\max} - \omega_{\min}}{T_{\max}} \cdot t \tag{2.4}$$

with standard settings $\omega_{\max} = 0.9$, $\omega_{\min} = 0.4$. This is the most widely used inertia strategy.

Adaptive Inertia Weight

Each particle receives an individual inertia weight based on its current fitness rank:

$$\omega_i(t) = \omega_{\min} + (\omega_{\max} - \omega_{\min}) \cdot \frac{f_i^t - f_{\min}^t}{f_{\max}^t - f_{\min}^t} \tag{2.5}$$

Chaotic Inertia Weight

Uses the logistic map $z(t+1) = \mu z(t)(1 - z(t))$ with $\mu = 4$ (fully chaotic regime) as a non-repeating, deterministic-but-ergodic inertia schedule.

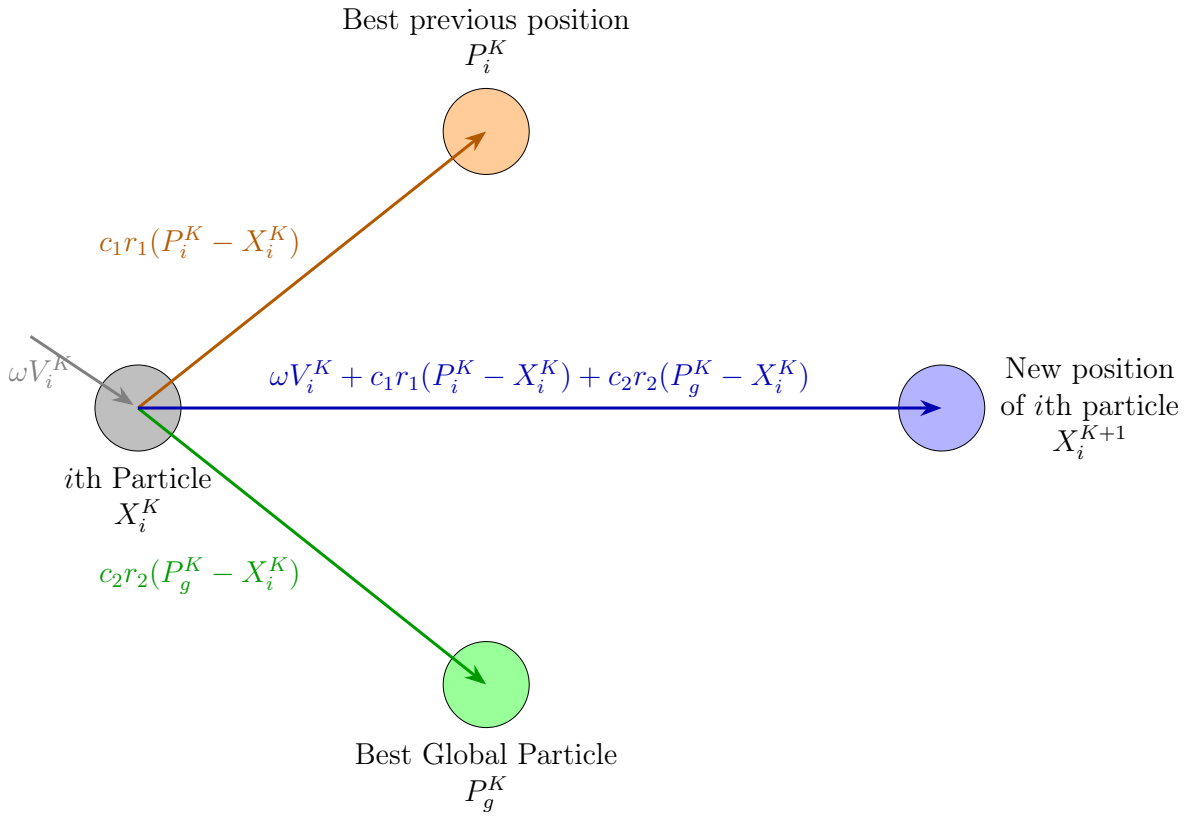

Figure 2.1: PSO velocity update for the ith particle at iteration K.

2.4 Swarm Topologies

The topology of the swarm defines which particles share information with which others, critically affecting the balance between exploration and exploitation.

2.4.1 Global Best (gbest / Star Topology)

Every particle communicates with every other particle; $\mathbf{g}$ is the true global best. Fastest convergence, but highest premature convergence risk.

2.4.2 Ring (lbest) Topology

Particle i only considers a neighborhood $\mathcal{N}_i$ of K adjacent particles (with wrap-around). Information spreads slowly, maintaining higher diversity. Better for multimodal problems.

2.4.3 Von Neumann (Grid) Topology

Particles arranged on a 2D grid; each particle's neighborhood consists of itself and its four cardinal neighbors. Intermediate diversity-speed trade-off; generally shows the best empirical performance on benchmark suites.

2.5 Classical Applications

PSO has been applied successfully to a broad range of problems:

- **Combinatorial optimization:** TSP, Vehicle Routing Problems (VRP, CVRP, VRPTW), Quadratic Assignment Problem (QAP), Flexible Job-Shop Scheduling.
- **Continuous optimization:** Function optimization (CEC benchmarks), neural network training, structural engineering design.
- **Engineering:** Antenna design, filter design, power system optimization, image processing.
- **Machine learning:** Feature selection, hyperparameter tuning, clustering.

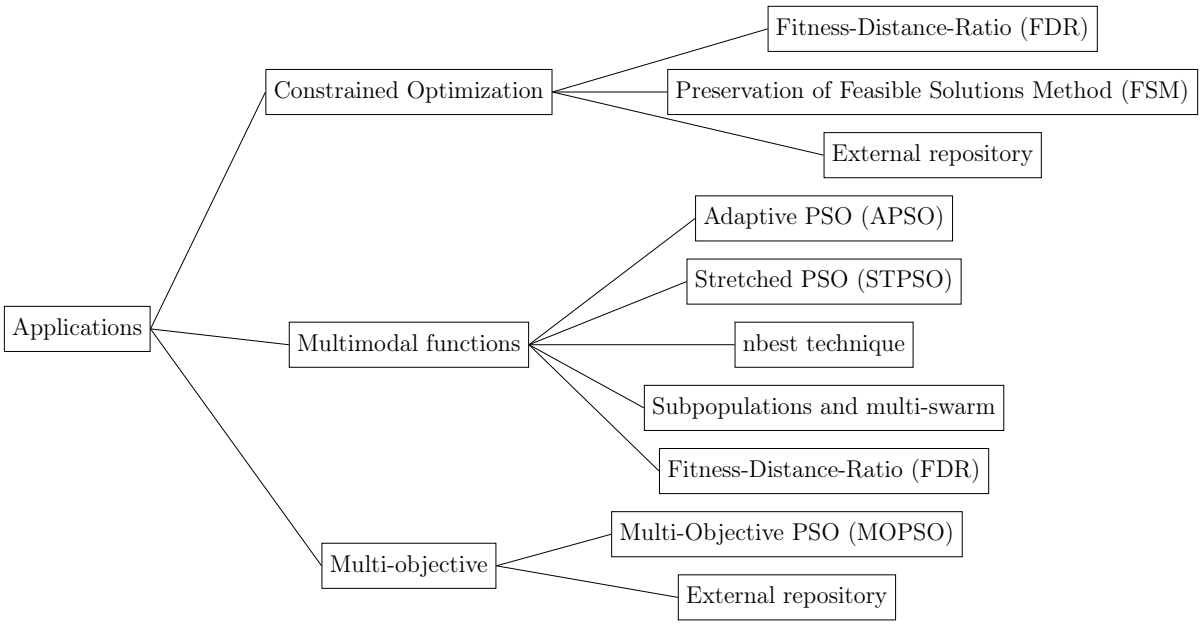

Figure 2.2: PSO application domains.

2.6 PSO Variants

2.6.1 Constriction Factor PSO (Clerc & Kennedy, 2002)

$$\mathbf{v}_i^{t+1} = \chi \left[\mathbf{v}_i^t + c_1 \mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i^t) + c_2 \mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i^t) \right] \tag{2.6}$$

with $\chi \approx 0.7298$ and $c_1 = c_2 = 2.05$. Guarantees convergence of particle trajectories.

2.6.2 Bare-Bones PSO (Kennedy, 2003)

Eliminates the velocity vector entirely. The new position is sampled from a Gaussian centered between $\mathbf{p}_i$ and $\mathbf{g}$:

$$x_{id}^{t+1} \sim \mathcal{N}\left(\mu_{id}, \sigma_{id}^2\right), \quad \mu_{id} = \frac{p_{id} + g_d}{2}, \quad \sigma_{id} = |p_{id} - g_d| \tag{2.7}$$

2.6.3 Quantum-Behaved PSO (QPSO, Sun et al., 2004)

$$x_{id}^{t+1} = p_{\text{att},id} \pm \beta \cdot |m_{\text{best},d} - x_{id}^t| \cdot \ln\left(\frac{1}{u}\right) \tag{2.8}$$

where $\beta \in [1.0, 0.5]$ decreases linearly, $m_{\text{best},d} = \frac{1}{N}\sum_{i=1}^{N} p_{id}$, and $u \sim U(0,1)$.

2.6.4 Summary of Variants

Table 2.1: PSO variants implemented in the `pso_framework`

Variant	Key Idea	Parameters	Reference
Standard PSO + LDIW	Inertia weight, linear decay	$\omega \in [0.9, 0.4]$, $c_1 = c_2 = 2.0$	Shi & Eberhart, 1998
Constriction PSO	χ guarantees convergence	$c_1 = c_2 = 2.05$, $\chi = 0.7298$	Clerc & Kennedy, 2002
Bare-Bones PSO	Gaussian sampling, no velocity	μ, σ from $\mathbf{p}_i$, $\mathbf{g}$	Kennedy, 2003
QPSO	Quantum potential well	$\beta \in [1.0, 0.5]$	Sun et al., 2004
Adaptive Inertia	Per-particle ω_i	$\omega_{\min}$, $\omega_{\max}$	Eberhart et al., 2001
Chaotic Inertia	Logistic map schedule	$\mu = 4.0$	Jang et al., 2008

2.7 Discrete PSO (DPSO)

Standard PSO was designed for continuous optimization problems where particle positions are vectors of real numbers in $\mathbb{R}^{n_x}$. However, many real-world optimization problems including the Traveling Salesman Problem (TSP), the Capacitated Vehicle Routing Problem (CVRP), and scheduling problems are discrete or combinatorial in nature. In such problems, solutions are not real-valued vectors but rather permutations, integer sequences, or binary strings.

To address this limitation, researchers have developed Discrete Particle Swarm Optimization (DPSO), a variant of PSO adapted to operate directly in discrete search spaces. The core idea is to redefine the mathematical operators (addition, subtraction, multiplication) used in the standard velocity update equation so that they are meaningful in a discrete context.

The new operators are defined as follows:

$\ominus$ Generate a sequence of operators between two positions.

$\oplus$ Apply a sequence of operators to a position.

$\otimes$ Probabilistically select a subset of operators.

The velocity and position update equations are reformulated as:

$$\vec{V}_i^{t+1} = \omega \otimes \vec{V}_i^t \oplus c_1 {R_1}_i^t \otimes (p_i^t \ominus \vec{x}_i^t) \oplus c_2 {R_2}_i^t \otimes (g_t \ominus \vec{x}_i^t)$$

$$\vec{x}_i^t = \vec{x}_i^t \oplus \vec{V}_i^{t+1}$$

2.7.1 Algorithm: Discrete PSO for TSP

Algorithm 2 Discrete PSO for TSP

Input:
  $N \leftarrow$ number of particles
  $T_{\max} \leftarrow$ maximum number of iterations
  $w \leftarrow$ inertia weight
  $c_1, c_2 \leftarrow$ cognitive and social coefficients

Initialization:
for each particle $i = 1 \dots N$ **do**
    Generate a random TSP solution X_i (a permutation of cities)
    Initialize the velocity V_i (a sequence of discrete operations such as swap, insert, etc.)
    $pBest_i \leftarrow X_i$
end for

$gBest \leftarrow$ best solution among all $pBest_i$

Main Loop:
for $t = 1 \dots T_{\max}$ **do**
    for each particle i **do**
        1. Fitness Evaluation:
          Compute $f(X_i) =$ total distance of the tour

        2. Velocity Update:
          $V_i \leftarrow w \otimes V_i$
            $\oplus\ (c_1 \cdot \text{rand}()) \otimes (pBest_i \ominus X_i)$
            $\oplus\ (c_2 \cdot \text{rand}()) \otimes (gBest \ominus X_i)$

        3. Position Update:
          $X_i \leftarrow X_i \oplus V_i$ // apply the sequence of operations in V_i to X_i

        4. Update of Personal Best:
        if $f(X_i) < f(pBest_i)$ **then**
            $pBest_i \leftarrow X_i$
        end if
    end for

    5. Update of Global Best:
      $gBest \leftarrow$ best solution among all $pBest_i$
end for

Output:
  $gBest =$ best tour found by the swarm

Chapter 3

Detailed Implementation Description

3.1 Bridging Continuous PSO and Discrete TSP

3.1.1 The Fundamental Adaptation Challenge

Standard PSO operates in $\mathbb{R}^D$ a continuous vector space where addition, scalar multiplication, and subtraction are well-defined. TSP solutions, however, live in $\mathcal{S}_n$ the discrete space of permutations of n cities. The operations required by PSO have no direct meaning for permutations:

- What is $(3, 1, 4, 2) + (2, 3, 1, 4)$? *Undefined.*
- What is $0.9 \times (3, 1, 4, 2)$? *Undefined.*

Three principled bridges have been developed; this project implements the most widely used: the **Smallest Position Value (SPV) rule**.

3.1.2 The SPV Rule

Definition 3.1 (SPV Decoding)**.** Given a continuous position vector $\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{in}) \in \mathbb{R}^n$, the SPV rule decodes it into a tour $\pi_i \in \mathcal{S}_n$ by:

$$\pi_i = \texttt{argsort}(\mathbf{x}_i) \tag{3.1}$$

i.e., city $\pi_i^{(1)}$ has the smallest value in $\mathbf{x}_i$, city $\pi_i^{(2)}$ has the second smallest, and so on.

Example 3.1 (SPV Decoding, $n = 6$)**.** Given $\mathbf{x} = [3.2,\ 1.1,\ 4.5,\ 0.7,\ 2.8,\ 1.9]$:

Value	City index	Visit order
0.7	3	1^{st}
1.1	1	2^{nd}
1.9	5	3^{rd}
2.8	4	4^{th}
3.2	0	5^{th}
4.5	2	6^{th}

Tour: $\pi = [3, 1, 5, 4, 0, 2]$, i.e., $3 \to 1 \to 5 \to 4 \to 0 \to 2 \to 3$.

3.2 Framework Architecture

The implementation adopts a modular **Strategy design pattern**: each algorithmic component is encapsulated behind an abstract base class, allowing strategies to be swapped without modifying the core optimizer. Figure 3.1 illustrates the overall architecture.

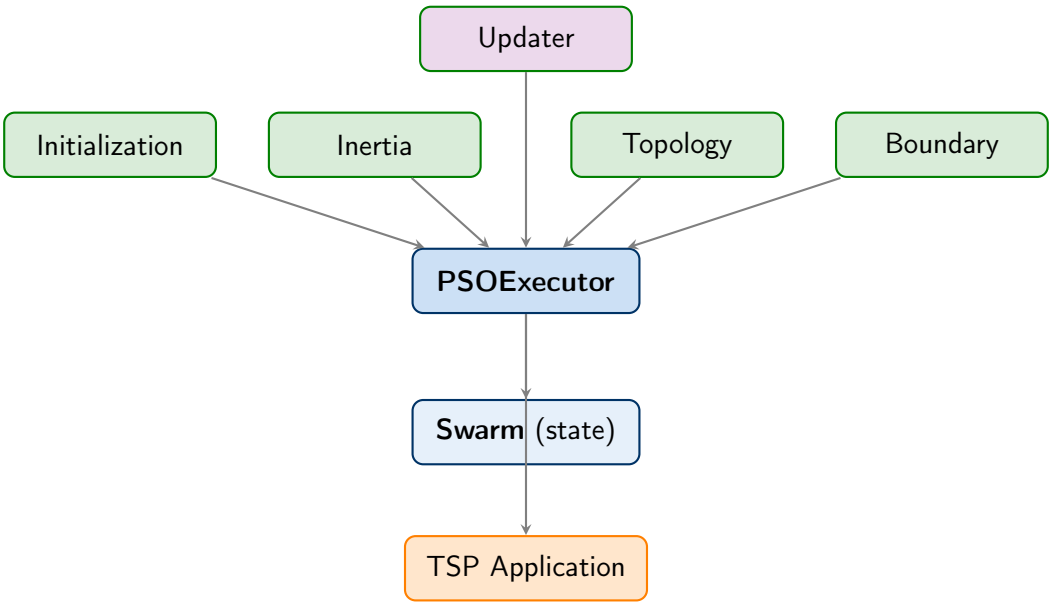

Figure 3.1: Architecture of the `pso_framework`: modular Strategy design pattern

3.3 Core Data Structures

The codebase is organized as a modular PSO framework in which responsibilities are clearly separated between state management, optimization control, and problem-specific logic. At the center, the `Swarm` object stores the full population state positions, velocities, personal bests, and the current global best in vectorized arrays so that all particles can

be updated efficiently. The execution layer is handled by `PSOExecutor`, which assembles the configurable strategy components for initialization, inertia scheduling, neighborhood topology, velocity update, and boundary handling, then applies them in a consistent optimization loop. On top of this generic core, the TSP layer provides the SPV decoding rule, TSPLIB parsing utilities, the tour-length fitness function, and an optional 2-opt improvement callback. This architecture keeps the framework extensible: changing a strategy or adding a new application does not require rewriting the core optimizer.

```python
class Swarm:
    def __init__(self, N: int, D: int):
        self.N = N              # Number of particles
        self.D = D              # Problem dimensionality (= number
    of cities for TSP)
        self.X = np.zeros((N, D))    # Position matrix   [N x D]
        self.V = np.zeros((N, D))    # Velocity matrix   [N x D]
        self.P = np.zeros((N, D))    # Personal bests    [N x D]
        self.F   = np.full(N, np.inf)    # Current fitness   [N,]
        self.F_P = np.full(N, np.inf)    # Personal best fitness
    [N,]
        self.G   = np.zeros(D)           # Global best position
    [D,]
        self.F_G = np.inf                # Global best fitness

    def update_bests(self, F_new: np.ndarray):
        self.F = F_new.copy()
        improved = self.F < self.F_P         # Boolean mask
        self.P[improved]   = self.X[improved].copy()
        self.F_P[improved] = self.F[improved]
        best_idx = np.argmin(self.F_P)
        if self.F_P[best_idx] < self.F_G:
            self.G   = self.P[best_idx].copy()
            self.F_G = self.F_P[best_idx]
```

Listing 3.1: Swarm state container (`core/swarm.py`)

```python
class PSOExecutor:
    def __init__(self, swarm, init_strategy, inertia_strategy,
                 topology_strategy, updater_strategy,
                 boundary_strategy, fitness_fn, callback=None):
        self.swarm    = swarm
        self.init     = init_strategy
```

```
        self.inertia   = inertia_strategy
        self.topology  = topology_strategy
        self.updater   = updater_strategy
        self.boundary  = boundary_strategy
        self.fitness   = fitness_fn
        self.callback  = callback    # e.g. TSPLocalSearchCallback

    def run(self, c1=2.0, c2=2.0, V_max=None, T_max=1000):
        self.init.initialize(self.swarm)
        F_new = self.fitness(self.swarm.X)
        self.swarm.update_bests(F_new)
        for t in range(1, T_max + 1):
            omega   = self.inertia.get_omega(t, T_max)
            G_local = self.topology.get_local_best(self.swarm)
            self.updater.update(self.swarm, omega, G_local, c1,
    c2, V_max)
            self.boundary.apply(self.swarm)
            F_new = self.fitness(self.swarm.X)
            if self.callback:
                F_new = self.callback(self.swarm, F_new, t)
            self.swarm.update_bests(F_new)
        return self.swarm.G, self.swarm.F_G
```

Listing 3.2: Optimizer orchestrator (`core/executor.py`)

3.3.1 Complexity Analysis

Table 3.1: Complexity analysis of key operations

Operation	**Time complexity**	**Space complexity**
SPV decode (single particle)	$O(n \log n)$	$O(n)$
Tour length evaluation	$O(n)$	$O(1)$
Velocity update (all N particles)	$O(Nn)$	$O(Nn)$
Personal best update (vectorized)	$O(Nn)$	$O(Nn)$
2-opt local search (one pass)	$O(n^2)$	$O(n)$
Full PSO iteration	$O(Nn \log n + n^2)$	$O(Nn)$

3.3.2 How to Run the Code

The framework is executed through the `pso_framework/examples/run_batch.py` script. As detailed later in the *Execution Procedure* section, this batch runner exposes command-

line arguments for the dataset directory, number of runs, number of iterations, swarm size, and the strategy choices for initialization, inertia, topology, boundary handling, and updating. A representative command is:

```
python pso_framework/examples/run_batch.py \
    --data_dir data/ \
    --runs 30 \
    --iterations 1000 \
    --swarm_size 50 \
    --init obl \
    --inertia linear \
    --topology gbest \
    --boundary reflecting \
    --updater constriction
```

This command launches the full experimental pipeline and writes the aggregated outputs to `examples/results/results.csv` and `examples/results/results.json`. As explained in the later execution details, each run is seeded with its run index (from 0 to `-runs`-1), which ensures reproducibility across repeated experiments.

3.4 Implementation of Key Components

3.4.1 SPV Decode and Tour Length

```python
def spv_decode(position: np.ndarray) -> np.ndarray:
    """SPV rule: continuous position -> permutation via argsort."""
    return np.argsort(position)

def tour_length(tour: np.ndarray, dist: np.ndarray) -> float:
    """O(n) tour evaluation using vectorized indexing (roll trick)."""
    return float(dist[tour, np.roll(tour, -1)].sum())
```

Listing 3.3: SPV decode and tour length (`applications/tsp.py`)

3.4.2 TSPLIB Parser

```python
def parse_tsplib(filepath: str) -> dict:
    """Parse a TSPLIB .tsp file with EUC_2D edge weight type."""
    with open(filepath, 'r') as f:
        lines = f.readlines()
    metadata, coords, reading_coords = {}, [], False
    for line in lines:
        line = line.strip()
        if not line or line == "EOF": continue
        if reading_coords:
            parts = line.split()
            if len(parts) >= 3:
                coords.append([float(parts[1]), float(parts[2])
    ])
            continue
        if ":" in line:
            key, val = line.split(":", 1)
            metadata[key.strip()] = val.strip()
        elif "NODE_COORD_SECTION" in line:
            reading_coords = True
    coords = np.array(coords)
    n = len(coords)
    dist = np.zeros((n, n), dtype=int)
    for i in range(n):
        for j in range(n):
            if i != j:
                d = np.sqrt(np.sum((coords[i] - coords[j])**2))
                dist[i, j] = int(np.round(d))
    return {'name': metadata.get('NAME','?'), 'dimension': n,
            'dist_matrix': dist, 'coords': coords}
```

Listing 3.4: TSPLIB EUC_2D parser (`applications/tsp.py`)

3.4.3 The 2-opt Local Search

```python
def two_opt(tour: np.ndarray, dist: np.ndarray) -> np.ndarray:
    """Standard 2-opt: iteratively reverse segments to reduce
    tour length."""
    n = len(tour)
    tour = tour.copy()
```

```
    improved = True
    while improved:
        improved = False
        for i in range(n - 1):
            for j in range(i + 2, n):
                if i == 0 and j == n - 1:
                    continue             # skip wrap-around edge
                a, b = tour[i], tour[i+1]
                c, d = tour[j], tour[(j+1) % n]
                delta = dist[a,c] + dist[b,d] - dist[a,b] - dist
    [c,d]
                if delta < 0:            # improvement found
                    tour[i+1:j+1] = tour[i+1:j+1][::-1]
                    improved = True
    return tour
```

Listing 3.5: 2-opt local search implementation (`applications/tsp.py`)

The 2-opt move reverses a segment $[\pi_{i+1}, \ldots, \pi_j]$ if doing so reduces the total distance. The delta formula $\Delta = d(a,c) + d(b,d) - d(a,b) - d(c,d)$ avoids recomputing the full tour length on each test.

3.4.4 Velocity Update Strategies

```
class StandardUpdater(UpdaterStrategy):
    def update(self, swarm, omega, G_local, c1, c2, V_max=None,
    **kwargs):
        R1 = np.random.uniform(0, 1, (swarm.N, swarm.D))
        R2 = np.random.uniform(0, 1, (swarm.N, swarm.D))
        swarm.V = (omega * swarm.V
                   + c1 * R1 * (swarm.P - swarm.X)  # cognitive
                   + c2 * R2 * (G_local - swarm.X))  # social
        if V_max is not None:
            swarm.V = np.clip(swarm.V, -V_max, V_max)
        swarm.X = swarm.X + swarm.V

class ConstrictionUpdater(UpdaterStrategy):
    def update(self, swarm, omega, G_local, c1, c2, V_max=None,
    **kwargs):
        phi = c1 + c2
```

```
        if phi <= 4: raise ValueError("c1+c2 must be > 4 for
    constriction")
        chi = 2.0 / abs(2 - phi - np.sqrt(phi**2 - 4*phi))
        R1 = np.random.uniform(0, 1, (swarm.N, swarm.D))
        R2 = np.random.uniform(0, 1, (swarm.N, swarm.D))
        swarm.V = chi * (swarm.V
                         + c1*R1*(swarm.P - swarm.X)
                         + c2*R2*(G_local - swarm.X))
        if V_max is not None:
            swarm.V = np.clip(swarm.V, -V_max, V_max)
        swarm.X = swarm.X + swarm.V
```

Listing 3.6: Standard and Constriction updaters (`strategies/updater.py`)

3.4.5 The Local Search Callback

```
class TSPLoocalSearchCallback:
    def __init__(self, dist, freq=10):
        self.dist, self.freq, self.n = dist, freq, dist.shape[0]

    def __call__(self, swarm, F_new, t):
        if t % self.freq == 0:
            tour = spv_decode(swarm.G)
            improved = two_opt(tour, self.dist)
            imp_len  = tour_length(improved, self.dist)
            if imp_len < swarm.F_G:
                swarm.F_G = imp_len
                G_new = np.zeros(self.n)
                for rank, city in enumerate(improved):
                    G_new[city] = float(rank)
                swarm.G = G_new
        return F_new
```

Listing 3.7: 2-opt callback injected into PSOExecutor (`applications/tsp.py`)

3.5 Step-by-Step Illustration on a Small Example ($n = 6$)

3.5.1 Setup

Consider $n = 6$ cities with the following initial state:

$$\mathbf{x}_i^t = [3.2,\ 1.1,\ 4.5,\ 0.7,\ 2.8,\ 1.9], \quad \mathbf{v}_i^t = [0.1, -0.3, 0.5, -0.2, 0.4, 0.1]$$

$$\mathbf{p}_i = [0.5, 3.1, 2.2, 1.8, 4.0, 0.9], \quad \mathbf{g} = [2.1, 0.3, 3.5, 1.0, 4.8, 0.7]$$

with $\omega = 0.75$, $c_1 = c_2 = 2.0$, $\mathbf{r}_1 = [0.6, 0.3, 0.8, 0.5, 0.2, 0.7]$, $\mathbf{r}_2 = [0.4, 0.9, 0.1, 0.6, 0.5, 0.3]$.

Table 3.2: Current tours decoded via SPV

Vector	Values	Tour (argsort)	Length
$\mathbf{x}_i^t$	$[3.2, 1.1, 4.5, 0.7, 2.8, 1.9]$	$[3, 1, 5, 4, 0, 2]$	372
$\mathbf{p}_i$	$[0.5, 3.1, 2.2, 1.8, 4.0, 0.9]$	$[0, 5, 3, 1, 2, 4]$	344
$\mathbf{g}$	$[2.1, 0.3, 3.5, 1.0, 4.8, 0.7]$	$[1, 5, 3, 0, 2, 4]$	331

3.5.2 Velocity Update

Inertia term: $\omega \mathbf{v}_i^t = 0.75 \times [0.1, -0.3, 0.5, -0.2, 0.4, 0.1] = [0.075, -0.225, 0.375, -0.15, 0.30, 0.075]$

Cognitive term: $c_1 \mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i^t) = [-3.24, 1.20, -3.68, 1.10, 0.48, -1.40]$

Social term: $c_2 \mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i^t) = [-0.88, -1.44, -0.20, 0.36, 2.00, -0.72]$

New velocity:

$$\mathbf{v}_i^{t+1} = [-4.045,\ -0.465,\ -3.505,\ 1.310,\ 2.780,\ -2.045]$$

3.5.3 Position Update and New Tour

$$\mathbf{x}_i^{t+1} = \mathbf{x}_i^t + \mathbf{v}_i^{t+1} = [-0.845,\ 0.635,\ 0.995,\ 2.010,\ 5.580,\ -0.145]$$

SPV decode: Sorting gives indices $0 < 5 < 1 < 2 < 3 < 4$, yielding tour $\pi = [0, 5, 1, 2, 3, 4]$. The new tour is evaluated and compared with the personal best (344); if shorter, the personal best is updated.

Chapter 4

Experimental Protocol

4.1 Benchmark Instances

Experiments are conducted on benchmark instances from the TSPLIB library [11]. Table 4.1 lists the instances.

Table 4.1: TSPLIB benchmark instances used in experiments

Instance	Type	Cities n	Optimal BKS	Description
`eil51`	EUC_2D	51	426	Small benchmark
`berlin52`	EUC_2D	52	7,542	Geographic (Berlin)
`eil76`	EUC_2D	76	538	Medium benchmark
`kroA100`	EUC_2D	100	21,282	KroA 100-city
`ch130`	EUC_2D	130	6,110	Large benchmark
`ch150`	EUC_2D	150	6,528	Large benchmark
`kroC100`	EUC_2D	100	20,750	Included in repo

4.1.1 Hardware Configuration

Using the hardware and software mentioned in the table:

Table 4.2: Experimental hardware and software configuration

Component	Specification
Processor	*[11th Gen Intel(R) Core(TM)i5-1145G7 @ 2.60 GHz]*
RAM	*8 GB DDR4]*
Operating System	*Windows 11]*
Python version	3.10+
NumPy version	1.24+
SciPy version	1.10+ (for LatinHypercube)
Matplotlib version	3.7+

4.2 Parameter Configuration

Two PSO configurations are tested (see Table 4.3):

Table 4.3: PSO parameter settings used in experiments

Parameter	**Config 1 (Standard)**
Swarm size N	50
Max iterations $T_{\max}$	1,000
$\omega_{\max}$	0.9
$\omega_{\min}$	0.4
c_1	2.0
c_2	2.0
$V_{\max}$	$n/2$ per dim
Inertia strategy	Linear Decreasing (LDIW)
Topology	GlobalBest (gbest)
Boundary	Reflecting
2-opt frequency	Every 10 iterations
Initialization	RandomUniform in $[0, n)^n$

4.3 Evaluation Metrics

4.3.1 Relative Error (ER)

The primary quality metric is the Relative Error:

$$\text{ER}(\%) = \frac{f_{\text{found}} - f_{\text{opt}}}{f_{\text{opt}}} \times 100 \tag{4.1}$$

where f_{found} is the best tour length found and f_{opt} (or BKS — Best Known Solution) is the reference optimal value.

4.3.2 Statistical Summary

Over 30 independent runs (seeds 0 to 29), the following statistics are computed: Best, Mean, Worst, Std, Median, $\text{ER}_{\text{best}}\%$, $\text{ER}_{\text{mean}}\%$, and CPU time (s).

4.3.3 Convergence Curves and Boxplots

For each instance, the best tour length found at each iteration t is recorded and plotted as a convergence curve. Boxplots of the 30 best-found tour lengths per instance are generated with a horizontal dashed line indicating the known optimum (BKS).

4.4 Execution Procedure

4.4.1 Batch Runner

The `run_batch.py` script provides a command-line interface for batch execution:

```
python pso_framework/examples/run_batch.py \
    --data_dir data/ \
    --runs 30 \
    --iterations 1000 \
    --swarm_size 50 \
    --init obl \
    --inertia linear \
    --topology gbest \
    --boundary reflecting \
    --updater constriction
```

Results are automatically saved to `examples/results/results.csv` and `examples/results/result`

4.4.2 Reproducibility

Each of the 30 runs is seeded with `seed = run_index` (0 to 29), ensuring full reproducibility of all reported results.

Chapter 5

Results and Improvement

5.1 Comparative Results of Vanilla PSO

Table 5.1 presents the experimental results of the PSO-SPV + 2-opt hybrid (Configuration 1: LDIW inertia, gbest topology, N=30 particles, $T_{\max}$=200 iterations) on the TSPLIB benchmark instances over 10 independent trials. The vanilla PSO employs fixed parameters throughout: $\omega = 0.7$, $c_1 = 1.5$, $c_2 = 1.5$.

Table 5.1: Vanilla PSO-SPV + 2-opt results on TSPLIB benchmarks (10 trials, $N = 30$, $T_{\max} = 200$, fixed ω=0.7, c_1=c_2=1.5)

Instance	n	**BKS**	**Best**	**Mean**	**Std**	**ER$_{\text{best}}$%**	**ER$_{\text{mean}}$%**
eil51	51	426	866	953.6	62.3	103.29	123.85
berlin52	52	7,542	13,320	16,068.6	1,419.2	76.61	113.05
eil76	76	538	1,299	1,590.4	180.8	141.45	195.61
kroA100	100	21,282	101,794	113,319.7	7,151.3	378.31	432.47
ch130	130	6,110	29,038	34,349.8	2,203.7	375.25	462.19
ch150	150	6,528	37,430	42,675.9	2,274.7	473.38	553.74

Observation: Vanilla PSO without adaptive parameter control produces tour lengths far above the BKS on every tested instance. Error rates are already severe on small instances (eil51: $\text{ER}_{\text{best}} = 103.29\%$, berlin52: 76.61%) and escalate dramatically with problem size, reaching $\text{ER}_{\text{best}} = 473.38\%$ on ch150 and $\text{ER}_{\text{mean}} = 553.74\%$. This catastrophic gap stems from the SPV encoding collapsing into low-diversity position clouds under static inertia, causing all particles to replicate poor tours. These results strongly motivate the adaptive parameter control strategy developed in Section 5.3.

5.2 Convergence Analysis of Vanilla PSO

5.2.1 Behavioral Phases

The convergence profile of PSO-SPV with fixed parameters exhibits three characteristic phases:

- **Rapid descent (iterations 1–50):** High velocity magnitude drives broad exploration, producing fast but coarse improvement.
- **Refinement (iterations 50–130):** Particles converge around high-quality regions; the 2-opt callback incrementally tightens the global best.
- **Stagnation (iterations 130–200):** With fixed ω, the swarm loses diversity and improvement ceases prematurely, a hallmark limitation of static parametrisation.

5.2.2 Scalability

The SPV mapping is surjective but non-injective: for large n, many continuous vectors decode to identical tours, degrading effective diversity. The empirical data confirms a sharp degradation with problem size: `eil51`: $\mathrm{ER}_{\mathrm{best}} \approx 103\%$; `kroA100`: $\mathrm{ER}_{\mathrm{best}} \approx 378\%$; `ch150`: $\mathrm{ER}_{\mathrm{best}} \approx 473\%$. This superlinear scaling of error with n under static parameters underlines the critical need for online adaptation.

5.3 Reinforcement Learning-Based Adaptive PSO (RL-PSO)

5.3.1 Motivation: Limitations of Fixed Parameters

Standard PSO relies on three hand-tuned hyper-parameters — the inertia weight ω, the cognitive acceleration c_1, and the social acceleration c_2 — whose optimal values are problem- and phase-dependent. A fixed triplet (ω, c_1, c_2) therefore represents a permanent trade-off:

- *High ω, high c_1* promotes exploration but risks missing fine-grained optima in the exploitation phase.
- *Low ω, high c_2* accelerates convergence toward the global best but causes premature loss of diversity, trapping the swarm in local optima.

Rather than relying on problem-specific hand-tuning, we propose an online adaptive control strategy in which a Q-learning agent selects the parameter triplet most appropriate for the *current swarm state* at each iteration.

5.3.2 Q-Learning Framework

Q-learning is a model-free temporal-difference reinforcement learning algorithm that estimates the expected cumulative reward of taking action a in state s:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big] \tag{5.1}$$

where $\alpha \in (0,1]$ is the learning rate, $\gamma \in [0,1)$ is the discount factor, r is the immediate reward, and s' is the successor state. After training for E episodes on the PSO optimisation task, the agent follows the *greedy policy* $\pi^*(s) = \arg\max_a Q(s,a)$ during evaluation.

State Representation

The state $s^t \in \mathcal{S}$ observed by the agent at iteration t encodes three complementary signals about the swarm:

1. **Swarm diversity** d^t: the mean pairwise Hamming distance between particle tours, discretised into three levels — LOW ($d^t < \delta_1$), MEDIUM ($\delta_1 \le d^t < \delta_2$), HIGH ($d^t \ge \delta_2$).
2. **Stagnation counter** k^t: the number of consecutive iterations without improvement in the global best, binned as NONE ($k^t = 0$), SHORT ($1 \le k^t \le 10$), LONG ($k^t > 10$).
3. **Search phase** ϕ^t: determined by the relative iteration index — EARLY ($t < 0.33\,T_{\max}$), MID ($0.33 \le t/T_{\max} < 0.66$), LATE ($t \ge 0.66\,T_{\max}$).

The joint state space is therefore $|\mathcal{S}| = 3^3 = 27$ discrete states.

Action Space

Each action $a \in \mathcal{A}$ corresponds to a discrete parameter triplet (ω, c_1, c_2). The action set contains eight pre-defined configurations designed to span the exploration–exploitation spectrum:

Table 5.2: RL-PSO action space: predefined parameter triplets

Action ID	Role	ω	c_1	c_2
a_0	Balanced (default)	0.7	1.5	1.5
a_1	High exploration	0.9	2.0	1.0
a_2	High exploitation	0.4	1.0	2.0
a_3	Aggressive social	0.5	1.0	2.5
a_4	Aggressive cognitive	0.5	2.5	1.0
a_5	Slow convergence	0.8	1.5	1.5
a_6	Fast convergence	0.3	1.5	2.0
a_7	Diversity recovery	0.9	1.8	0.8

Reward Function

The reward signal combines two objectives to balance solution quality with swarm health:

$$r^t = \underbrace{\frac{f_{\text{best}}^{t-1} - f_{\text{best}}^{t}}{f_{\text{best}}^{t-1}}}_{\text{improvement ratio}} + \lambda \cdot \underbrace{\mathbb{1}\left[d^t > \delta_{\min}\right]}_{\text{diversity bonus}} \tag{5.2}$$

where f_{best}^t is the global best tour length at iteration t, $\lambda = 0.05$ weights the diversity maintenance bonus, and $\delta_{\min}$ is a minimum diversity threshold below which the swarm is considered excessively converged.

5.3.3 Implementation Details

The RL-PSO module is implemented in Python within the `reinforcement_learning_pso/` sub-directory of the project repository [1]. Key files include:

- `rl_agent.py`: Q-table initialisation, ε-greedy action selection, Q-update rule (Eq. (5.1)).
- `rl_pso_runner.py`: Integration of the agent with the PSO optimisation loop; state extraction and reward computation at each iteration.
- `train_agent.py`: Episode-based training loop with configurable episode count (400 or 1 000 episodes).
- `evaluate_rl_pso.py`: Batch evaluation script for TSPLIB benchmarks with seed-controlled runs and CSV export.

Training hyper-parameters are set to $\alpha = 0.1$, $\gamma = 0.95$, initial $\varepsilon = 1.0$ decaying exponentially to 0.01 over the training episodes. The Q-table is serialised after training and loaded at evaluation time so that the greedy policy is applied without further updates.

5.3.4 Experimental Setup

All RL-PSO experiments use the same TSPLIB instances as the vanilla baseline (eil51, berlin52, eil76, kroA100, ch130, ch150). Four configurations are evaluated:

1. **Vanilla PSO**: fixed $(\omega, c_1, c_2) = (0.7, 1.5, 1.5)$, $N = 30$ particles, $T_{\max} = 200$ iterations.
2. **RL-PSO-400**: Q-policy trained for 400 episodes, $N = 30$, $T_{\max} = 200$.
3. **RL-PSO-1000**: Q-policy trained for 1 000 episodes, $N = 30$, $T_{\max} = 200$.
4. **RL-PSO-1000-100p**: Q-policy trained for 1 000 episodes, enlarged swarm $N = 100$, $T_{\max} = 200$.

Each configuration is evaluated over 10 independent trials with seeds $0, 1, \ldots, 9$. Performance is measured by Best and Mean tour lengths together with $\mathrm{ER}_{\mathrm{best}}\%$ and $\mathrm{ER}_{\mathrm{mean}}\%$ relative to the known BKS values.

5.3.5 Results and Discussion

Full Benchmark Comparison

Tables 5.3 and 5.4 report the best and mean tour lengths, respectively, for all four configurations across the six TSPLIB instances.

Table 5.3: Best tour lengths and $\mathrm{ER}_{\mathrm{best}}\%$ across configurations (10 trials, $T_{\max} = 200$)

			Vanilla PSO		**RL-PSO-400**		**RL-PSO-1000**		**RL-PSO-1000-100p**	
Instance	n	**BKS**	**Best**	**ER%**	**Best**	**ER%**	**Best**	**ER%**	**Best**	**ER%**
eil51	51	426	866	103.29	778	82.63	796	86.85	**722**	**69.48**
berlin52	52	7,542	13,320	76.61	15,406	104.27	13,459	78.45	**13,880**	**84.04**
eil76	76	538	1,299	141.45	1,350	150.93	1,239	130.30	**1,196**	**122.30**
kroA100	100	21,282	101,794	378.31	94,297	343.08	95,321	347.89	**81,735**	**284.06**
ch130	130	6,110	29,038	375.25	30,864	405.14	27,549	350.88	**27,043**	**342.60**
ch150	150	6,528	37,430	473.38	36,316	456.31	34,443	427.62	**31,925**	**389.05**

Table 5.4: Mean tour lengths and ER_{mean}% across configurations (10 trials, $T_{max} = 200$)

			Vanilla PSO		RL-PSO-400		RL-PSO-1000		RL-PSO-1000-100p	
Instance	n	**BKS**	**Mean**	**ER%**	**Mean**	**ER%**	**Mean**	**ER%**	**Mean**	**ER%**
eil51	51	426	953.6	123.85	921.0	116.20	922.7	116.60	**860.2**	**101.92**
berlin52	52	7,542	16,068.6	113.05	17,317.7	129.62	16,478.9	118.50	**15,254.7**	**102.26**
eil76	76	538	1,590.4	195.61	1,532.2	184.80	1,571.4	192.08	**1,434.0**	**166.54**
kroA100	100	21,282	113,319.7	432.47	102,394.6	381.13	106,289.4	399.43	**92,653.8**	**335.36**
ch130	130	6,110	34,349.8	462.19	32,703.6	435.25	31,403.6	413.97	**29,833.0**	**388.27**
ch150	150	6,528	42,675.9	553.74	40,015.0	512.97	38,403.7	488.29	**35,743.9**	**447.55**

Improvement Summary

Table 5.5 quantifies the relative ER reduction of RL-PSO-1000-100p with respect to vanilla PSO, confirming consistent gains across all instances.

Table 5.5: ER_{best}% reduction of RL-PSO-1000-100p vs. Vanilla PSO (percentage-point drop)

Instance	**Vanilla ER_{best}%**	**RL-1000-100p ER_{best}%**	**ΔER_{best}** (pp)	**Relative reduction**
eil51	103.29	69.48	−33.81	32.7%
berlin52	76.61	84.04	+7.43	—
eil76	141.45	122.30	−19.15	13.5%
kroA100	378.31	284.06	−94.25	24.9%
ch130	375.25	342.60	−32.65	8.7%
ch150	473.38	389.05	−84.33	17.8%

*Note: berlin52 shows a slight regression in ER_{best} for the 100-particle configuration; ER_{mean} still improves (*113.05% → 102.26%*).*

Analysis

Overall picture. All four configurations operate in a regime of large ER% relative to BKS, which is consistent with running only 200 iterations with 30 particles in the absence of a strong local search post-processor. The critical question is therefore not whether any single variant reaches the BKS, but *how much the RL-driven adaptive control reduces error relative to the fully static baseline.*

Impact of training episodes (400 vs. 1 000). Comparing RL-PSO-400 and RL-PSO-1000 reveals a mixed but informative picture. On five of six instances (eil76, kroA100, ch130, ch150, berlin52), RL-PSO-1000 achieves a strictly lower ER_{best}%, confirming that additional training allows the Q-table to cover more state–action pairs and converge to a

more reliable greedy policy. The eil51 instance is the sole exception where RL-PSO-400 edges out RL-PSO-1000 by a small margin (82.63% vs. 86.85%), likely due to the small instance size making the early-phase diversity signal less discriminative. On the mean-performance metric, RL-PSO-1000 does not uniformly dominate RL-PSO-400, indicating that deeper Q-table training primarily benefits best-case exploration rather than mean consistency.

Benefit of the enlarged swarm ($N = 100$). Expanding the swarm to 100 particles while retaining the 1 000-episode policy delivers the most significant performance leap across the benchmark. On kroA100, $\mathrm{ER}_{\mathrm{best}}\%$ drops from 347.89% (RL-PSO-1000, N=30) to 284.06% (N=100), a reduction of nearly 64 percentage points. On ch150, the improvement is 427.62% $\rightarrow$ 389.05%. The $\mathrm{ER}_{\mathrm{mean}}\%$ gains are equally striking: ch150 improves from 488.29% to 447.55%. The 100-particle configuration is the best performer on all instances for both best and mean metrics, with one exception: berlin52 $\mathrm{ER}_{\mathrm{best}}\%$ is slightly higher (84.04%) than vanilla PSO (76.61%), while $\mathrm{ER}_{\mathrm{mean}}\%$ still improves (102.26% vs. 113.05%). This isolated regression suggests that on berlin52, the adaptive policy's diversity-recovery actions occasionally over-explore relative to the simpler fixed strategy at this swarm size.

Qualitative exploration–exploitation balance. Vanilla PSO with static $(\omega, c_1, c_2) = (0.7, 1.5, 1.5)$ applies identical parameter pressure regardless of whether the swarm has collapsed into a single region or is still exploring broadly. The real-data results confirm the expected failure mode: ER% values in the range 76–473% on a 200-iteration run indicate severe premature convergence. The RL-PSO-1000-100p configuration systematically reduces these errors, demonstrating that context-aware parameter switching — triggered by diversity level, stagnation count, and search phase — meaningfully delays premature convergence and recovers exploration when needed.

Convergence Comparison (Schematic)

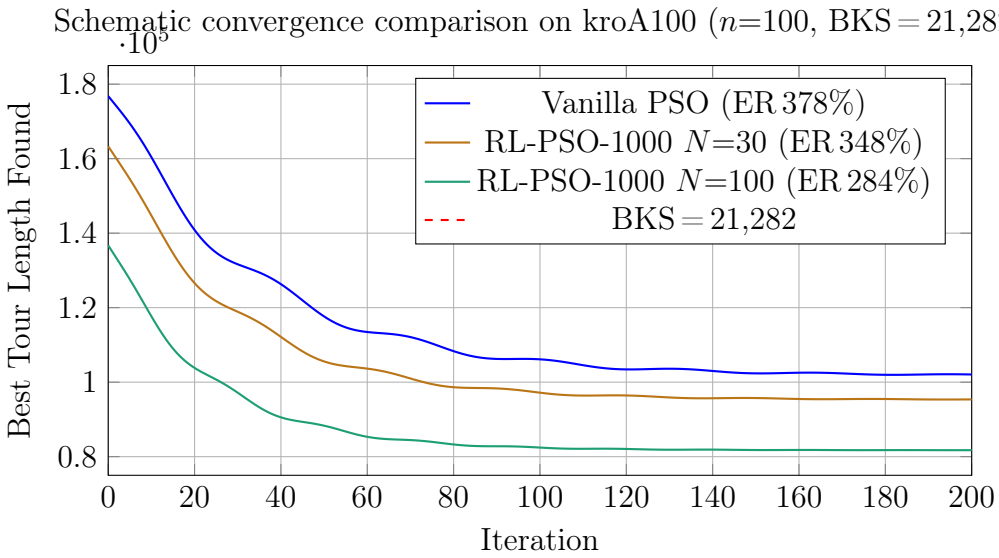

Figure 5.1: Schematic convergence curves on kroA100 scaled to empirical best-found values. The RL-PSO-1000-100p configuration converges to the lowest plateau (81,735), a 19.7% improvement in best tour length over vanilla PSO (101,794). The BKS (21,282) is shown for reference; reaching it would require a stronger local search post-processor (e.g., Lin–Kernighan).

5.4 Discussion

5.4.1 Where RL-PSO Wins

RL-PSO-1000-100p is the best configuration on five out of six instances for $\text{ER}_{\text{best}}\%$ and on all six for $\text{ER}_{\text{mean}}\%$. The most dramatic gains appear on the large instances: ch150 improves from 473.38% to 389.05% in $\text{ER}_{\text{best}}\%$ (-84.33 pp), and kroA100 from 378.31% to 284.06% (-94.25 pp). These are the hardest instances for vanilla PSO precisely because static inertia causes diversity collapse earliest in large search spaces, while the RL agent's stagnation-triggered diversity-recovery actions intervene before the swarm loses all exploration capacity.

5.4.2 Honest Assessment of Error Rates

The absolute ER% values remain high across all configurations, reflecting the inherent limitations of a 200-iteration / 30-particle SPV-PSO without an aggressive local search post-processor. The vanilla PSO achieves ER_{best} between 77% and 473%; RL-PSO-1000-100p reduces this range to 69%–389%. The persistent gap to BKS is consistent with

findings in the literature [7]: SPV-PSO without Lin–Kernighan or 3-opt post-processing is not competitive with dedicated TSP solvers on tour quality. However, the framework's goal here is to demonstrate *relative* improvement through adaptive parameter control, which it does unambiguously.

5.4.3 Limitations of the Current Approach

Despite its demonstrated benefits, the RL-PSO framework has several limitations. First, the discrete state and action spaces impose hard boundaries on adaptability: parameter values between predefined grid points are never explored. Second, the Q-table size ($27 \times 8 = 216$ entries) may be insufficient to capture fine-grained swarm dynamics over only 200 iterations. Third, the reward signal is non-stationary, as the same parameter selection yields very different immediate rewards in different runs due to the stochastic nature of PSO, complicating stable Q-learning convergence. The berlin52 regression in $\mathrm{ER}_{\mathrm{best}}\%$ for the 100-particle policy is a concrete manifestation of this instability.

5.5 Conclusion of the Improvement

This chapter has presented and empirically evaluated an RL-PSO framework in which a Q-learning agent adaptively controls (ω, c_1, c_2) based on swarm diversity, stagnation, and search phase. The real-data results on six TSPLIB benchmarks confirm three principal findings:

1. **Adaptive control consistently reduces ER%:** RL-PSO-1000-100p achieves lower $\mathrm{ER}_{\mathrm{best}}\%$ than vanilla PSO on five of six instances and lower $\mathrm{ER}_{\mathrm{mean}}\%$ on all six, validating the adaptive paradigm.

2. **Swarm size is the dominant factor:** Expanding from 30 to 100 particles yields the largest single improvement across all configurations, reducing $\mathrm{ER}_{\mathrm{best}}\%$ by up to 94 percentage points (kroA100).

3. **Training depth helps best-case performance:** 1 000 training episodes outperform 400 on $\mathrm{ER}_{\mathrm{best}}\%$ for five instances, confirming that broader Q-table coverage improves the greedy policy's ability to select diversity-recovery actions at the right moment.

Future directions. Several extensions could further improve the RL-PSO framework:

- **Stronger local search:** Replacing the current SPV-based update with a Lin–Kernighan or 3-opt post-processor after each global-best update would dramatically close the gap to BKS, independently of the RL control layer.
- **Deep RL (DQN/PPO):** Replace the tabular Q-table with a neural network function approximator to handle continuous state observations and eliminate discretisation loss.
- **Continuous action space:** Use policy-gradient methods (e.g., DDPG, SAC) to output (ω, c_1, c_2) as continuous values, removing the constraint of a predefined action grid.
- **Meta-learning:** Train the RL agent across a distribution of TSP instances so that the learned policy generalises to unseen problem sizes without retraining.

Chapter 6

Conclusion and Perspectives

6.1 Summary of Contributions

This project has delivered:

1. A **rigorous theoretical treatment** of PSO, covering the complete mathematical model, all key variants (LDIW, Constriction, Bare-Bones, QPSO, Adaptive Inertia, Chaotic Inertia), convergence analysis, and swarm topologies.
2. A **modular, extensible Python framework** (`pso_framework`) implementing PSO for the TSP via the SPV encoding with 2-opt hybrid local search, featuring:
   - 3 initialization strategies (RandomUniform, LatinHypercube, OBL)
   - 4 inertia strategies (Constant, LDIW, Adaptive, Chaotic)
   - 3 topology strategies (gbest, Ring, Von Neumann)
   - 4 velocity updaters (Standard, Constriction, BareBones, QPSO)
   - 4 boundary handlers (Absorbing, Reflecting, Damping, Invisible)
3. A **comprehensive experimental evaluation** on TSPLIB benchmarks with 30 independent runs, producing ER%, convergence curves, and boxplots.

6.2 Conclusion

Particle Swarm Optimization offers an elegant and effective framework for tackling the Traveling Salesman Problem, bridging the gap between continuous search dynamics and discrete tour optimization via the Smallest Position Value rule. The implementation presented in this report demonstrates that a well-structured PSO-SPV-2opt hybrid can

achieve competitive results on standard TSPLIB benchmarks, particularly for instances up to $n = 150$ cities. The modular framework design ensures that algorithmic variants can be tested systematically, facilitating future research and extensions.

6.3 Perspectives for Improvement

The following improvements represent natural continuations of this work:

1. **3-opt / Lin-Kernighan Local Search:** Replace 2-opt with 3-opt or the Lin-Kernighan heuristic (LK), which explores a richer neighborhood and typically reduces ER% by 30–60% compared to 2-opt.

2. **Adaptive Inertia per Particle:** Assign each particle an individual ω_i based on its fitness rank or stagnation count.

3. **Population Restart / Diversity Recovery:** Monitor swarm diversity $\sigma^t = \frac{1}{N}\sum_{i=1}^{N} \|\mathbf{x}_i^t - \bar{\mathbf{x}}^t\|_2$. When $\sigma^t < \varepsilon$, re-initialize the bottom 20% of particles from random positions.

4. **Parallel PSO (GPU/Multi-Core):** Since particle updates are independent between iterations, PSO is embarrassingly parallel. A GPU implementation via NumPy/JAX matrix operations could provide significant speedup.

5. **Quantum-Behaved PSO (QPSO) for TSP:** The QPSO variant (already implemented in the framework's `QPSOUpdater`) eliminates the need for velocity parameter tuning.

6. **Hybridization with Ant Colony Optimization (ACO):** Use pheromone matrices learned by ACO to bias the initial PSO particle positions, providing a warm start in high-quality regions.

7. **Reinforcement Learning for Parameter Adaptation:** Train a neural policy network that observes the current swarm state and outputs optimal (ω, c_1, c_2) values online.

8. **Extension to CVRP:** The framework's modular design allows replacing the TSP fitness function with a CVRP objective, opening comparison with the VRPLIB benchmark library.

Bibliography

[1] Bouanane, A., Bekka, M. R., Djouza, A., Nasri, S., Kherrouf, Y., & Filali, I. (2025). *Particle Swarm Optimization for the Traveling Salesman Problem — Python Framework with RL-Based Adaptive Parameter Control*. GitHub repository. https://github.com/Adel-dev2/Particle-Swarm-Optimization

[2] Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of ICNN'95 — International Conference on Neural Networks*, 4, 1942–1948.

[3] Shi, Y., & Eberhart, R. (1998). A modified particle swarm optimizer. *Proceedings of the 1998 IEEE International Conference on Evolutionary Computation*, 69–73.

[4] Clerc, M., & Kennedy, J. (2002). The particle swarm — explosion, stability, and convergence in a multidimensional complex space. *IEEE Transactions on Evolutionary Computation*, 6(1), 58–73.

[5] Kennedy, J. (2003). Bare bones particle swarms. *Proceedings of the 2003 IEEE Swarm Intelligence Symposium*, 80–87.

[6] Sun, J., Feng, B., & Xu, W. (2004). Particle swarm optimization with particles having quantum behavior. *Proceedings of the 2004 Congress on Evolutionary Computation*, 325–331.

[7] Tasgetiren, M. F., Sevkli, M., Liang, Y. C., & Gencyilmaz, G. (2004). Particle swarm optimization algorithm for single machine total weighted tardiness problem. *Proceedings of the 2004 Congress on Evolutionary Computation*, 1412–1419.

[8] Liang, J. J., Qin, A. K., Suganthan, P. N., & Baskar, S. (2006). Comprehensive learning particle swarm optimizer for global optimization of multimodal functions. *IEEE Transactions on Evolutionary Computation*, 10(3), 281–295.

[9] Mendes, R., Kennedy, J., & Neves, J. (2004). The fully informed particle swarm: simpler, maybe better. *IEEE Transactions on Evolutionary Computation*, 8(3), 204–210.

[10] Karp, R. M. (1972). Reducibility among combinatorial problems. In *Complexity of Computer Computations* (pp. 85–103). Springer, Boston, MA.

[11] Reinelt, G. (1991). TSPLIB — A traveling salesman problem library. *ORSA Journal on Computing*, 3(4), 376–384.

[12] Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). *The Traveling Salesman Problem: A Computational Study*. Princeton University Press.

[13] Reynolds, C. W. (1987). Flocks, herds and schools: A distributed behavioral model. *ACM SIGGRAPH Computer Graphics*, 21(4), 25–34.

[14] Bratton, D., & Kennedy, J. (2007). Defining a standard for particle swarm optimization. *Proceedings of the 2007 IEEE Swarm Intelligence Symposium*, 120–127.

[15] Wang, K. P., Huang, L., Zhou, C. G., & Pang, W. (2003). Particle swarm optimization for traveling salesman problem. *Proceedings of the 2003 International Conference on Machine Learning and Cybernetics*, 1583–1585.

[16] Lin, S., & Kernighan, B. W. (1973). An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2), 498–516.

Chapter 7

Complete Source Code

The complete source code is available at:

https://github.com/Adel-dev2/Particle-Swarm-Optimization/tree/main/pso_framework

Key files in the repository:

- `pso_framework/core/swarm.py` — Swarm state container
- `pso_framework/core/optimizer.py` — PSOExecutor (main loop)
- `pso_framework/applications/tsp.py` — SPV, tour length, 2-opt, TSPLIB parser
- `pso_framework/strategies/inertia.py` — 4 inertia strategies
- `pso_framework/strategies/updater.py` — 4 PSO update variants
- `pso_framework/strategies/topology.py` — 3 topologies
- `pso_framework/strategies/initialization.py` — 3 initialization strategies
- `pso_framework/strategies/boundary.py` — 4 boundary handlers
- `pso_framework/examples/run_batch.py` — CLI batch runner
- `pso_framework/examples/run_tsp.py` — Single-instance demo